

markpublish Benutzerhandbuch

Moderne PDF- und HTML-Dokumentenerstellung aus Markdown

Dieses Handbuch bietet eine praxisorientierte Anleitung und vollständige Referenz zur Konfiguration, Template-Anpassung und Dokumentengenerierung mit markpublish.

AUTOR
Frank Winter

STATUS
Freigegeben

VERSION
1.0.0

DATUM
01.09.2026

COPYRIGHT
© 2026 Frank Winter

Inhaltsverzeichnis

1 Einführung & Architektur	5
1.1 Motivation & Zielsetzung	5
1.2 Die Verarbeitungs-Pipeline	5
2 Installation & Schnellstart	7
2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Nachschlagen	8
2.5 Dokument erstellen	8
3 Die CLI-Befehle	10
3.1 Befehlsübersicht	10
3.2 markpublish init	10
3.3 markpublish cheatsheet	10
3.4 markpublish manual	11
3.5 markpublish build	12
3.6 markpublish templates	12
3.7 markpublish export-template	13
3.8 markpublish labels	13
4 Konfigurations-Leitfaden	16
4.1 Das document-Objekt	16
4.2 Kapitel definieren	16
4.3 Hierarchische Unterkapitel	17
4.4 Übergeordnete Abschnitte (Parts / Blöcke)	17
5 Das Template- und Design-System	20
5.1 Theme-basierte Verzeichnisstruktur	20
5.2 Die 3-stufige Auflösungs-Hierarchie	20
5.3 Statische Texte und Sprache	21
5.4 Kopf- und Fußzeilen	26
6 Markdown-Features & Syntax	30
6.1 Code-Blöcke & Syntax Highlighting	30
6.2 Tabellen	30

6.3 GitHub-Style Callout-Boxen (Alerts) 30

6.4 Tasklisten 31

Anhänge

Anhang A: YAML-Schema-Referenz 32

Anhang B: Fehlerbehebung & FAQ 38

KAPITEL 1

Einführung & Architektur

Überblick über Zielsetzung, Kernkonzepte und die modulare Verarbeitungs-Pipeline.

1 Einführung & Architektur

Willkommen beim **markpublish Benutzerhandbuch**. `markpublish` ist ein modernes, quelloffenes Publishing-Werkzeug, das aus einfachen Markdown-Dateien professionell gesetzte **PDF-Publikationen** und **HTML-Vorschauen** erzeugt.

1.1 Motivation & Zielsetzung

Viele Dokumentationswerkzeuge erfordern entweder komplexe LaTeX-Setups oder beschränken sich auf reine HTML-Webseiten. `markpublish` verbindet die Einfachheit von Markdown mit der typografischen Präzision von **CSS Paged Media** über die [WeasyPrint](#)-Engine.

1.1.1 Kernvorteile auf einen Blick:

- **Modulare Kapitel:** Jedes Kapitel wird als separate Markdown-Datei gepflegt.
- **Hierarchische Struktur:** Beliebig tief schachtelbare Unterkapitel und übergeordnete Abschnitte (*Parts*).
- **Deklarative Steuerung:** Ein zentrales Manifest (`markpublish.yaml`) steuert Inhalt, Metadaten und Layout.
- **W3C CSS Paged Media:** Exakte Kontrolle über `@page` -Ränder, mehrzeilige Kopf- und Fußzeilen, Seitenzahlen und Trennseiten.
- **Erweiterbare Pipeline:** Saubere Trennung zwischen Parser, Renderer und Template-Ebenen.

1.2 Die Verarbeitungs-Pipeline

Die Architektur von `markpublish` folgt einem mehrstufigen Prozess:

1. **Konfigurations-Lader (`config.loader`):** Liest die YAML-Struktur ein, validiert Datentypen und löst dynamische Variablen (z. B. `date: auto`) auf.
2. **Template-Resolver (`templates.resolver`):** Sucht nach der 3-stufigen Priorität (*User > Common/Workspace > Package*) nach dem passenden Theme für das Zielformat.
3. **Markdown- & TOC-Engine (`markdown.engine`):** Parst Markdown mit erweiterten Erweiterungen (Callouts, Tabellen, Pygments-Syntax-Highlighting), vergibt konsistente Überschriftennummern (`1.1` , `1.2`) und baut Inhaltsverzeichnisse auf.
4. **Renderer (`renderers.pdf` & `renderers.html`):** Übergibt die gerenderten HTML- und CSS-Fragmente an WeasyPrint für den PDF-Export oder speichert eigenständiges, responsives HTML.

KAPITEL 2

Installation & Schnellstart

Schritt-für-Schritt-Anleitung zur Installation und Erstellung des ersten Dokuments.

AUF EINEN BLICK

2.1 Voraussetzungen	7
2.2 Installation via pip	7
2.3 Erstes Projekt initialisieren	7
2.4 Nachschlagen	8
2.5 Dokument erstellen	8

2 Installation & Schnellstart

In diesem Kapitel erfahren Sie, wie Sie `markpublish` installieren und in weniger als zwei Minuten Ihr erstes Dokument kompilieren.

2.1 Voraussetzungen

`markpublish` benötigt **Python 3.10 oder neuer**. Es läuft nativ unter: - **Windows** (10, 11, Server) - **macOS** (Intel und Apple Silicon) - **Linux** (Debian, Ubuntu, Fedora, Arch, etc.)

2.2 Installation via pip

Installieren Sie das Paket über den Python-Paketmanager:

```
pip install markpublish
```

Für Entwickler oder zur Installation aus dem Quellcode:

```
git clone https://github.com/your-username/markpublish.git
cd markpublish
pip install -e ".*[dev]"
```

2.3 Erstes Projekt initialisieren

Verwenden Sie den Befehl `init`, um eine neue Dokumentenstruktur zu erzeugen:

```
markpublish init mein-leitfaden --title "Mein Leitfaden"
cd mein-leitfaden
```

Der Befehl legt einen bewusst minimalen Stumpf an — zwei Dateien, direkt im Zielordner:

```
mein-leitfaden/
├─ markpublish.yaml      # Das Dokumenten-Manifest
└─ next-steps.md        # Ein Kapitel, zum Ersetzen gedacht
```

2.4 Nachschlagen

```
# zweiseitige Kurzreferenz
markpublish cheatsheet [--lang de|en]

# dieses Handbuch
markpublish manual [--lang de|en]
```

Beide werden aus Quellen gerendert, die im Paket mitgeliefert werden — sie passen also immer zur installierten Version. Ein erfolgreicher Lauf ist zugleich der Nachweis, dass die Rendering-Kette funktioniert; unter Windows also, dass WeasyPrint seine GTK-Laufzeit findet.

Ohne `--lang` entscheidet die Sprache Ihres Systems; gibt es dafür keine Übersetzung, erscheint die englische. Der Parameter wählt die **Quelle** des Handbuchs, nicht bloß die Beschriftungen der Oberfläche.

2.5 Dokument erstellen

Rendern Sie Ihr Dokument mit dem `build`-Befehl:

```
# PDF aus dem aktuellen Projektverzeichnis erstellen
markpublish build

# HTML-Ausgabe erzeugen
markpublish build --target html

# Sowohl PDF als auch HTML in einem Durchgang bauen
markpublish build --target all
```

Die fertigen Dateien werden direkt im Projektordner abgelegt.

KAPITEL 3

Die CLI-Befehle

Alle Befehle im Detail: build, init, cheatsheet, manual, templates, export-template und labels.

AUF EINEN BLICK

3.1 Befehlsübersicht	10
3.2 markpublish init	10
3.3 markpublish cheatsheet	10
3.4 markpublish manual	11
3.5 markpublish build	12
3.6 markpublish templates	12
3.7 markpublish export-template	13
3.8 markpublish labels	13

3 Die CLI-Befehle

`markpublish` bietet eine schlanke, intuitive Befehlszeilenschnittstelle (CLI), die über `markpublish` oder die Kurzform `mpub` aufgerufen werden kann.

3.1 Befehlsübersicht

BEFEHL	KURZBESCHREIBUNG
<code>markpublish init</code>	Erzeugt einen minimalen Projektstumpf (zwei Dateien)
<code>markpublish cheatsheet</code>	Rendert die zweiseitige Kurzreferenz
<code>markpublish manual</code>	Rendert dieses Handbuch
<code>markpublish build</code>	Kompiliert das Dokument zu PDF und/oder HTML
<code>markpublish templates</code>	Listet alle gefundenen Templates und deren Quellen auf
<code>markpublish export-template</code>	Exportiert ein Template zur individuellen Anpassung
<code>markpublish labels</code>	Zeigt die aufgelösten statischen Texte und ihre Herkunft

3.2 `markpublish init`

Legt einen minimalen Projektstumpf an: eine `markpublish.yaml` und ein Kapitel `next-steps.md`, beide direkt im Zielordner. Kein `chapters/`-Verzeichnis, keine Beispielkapitel.

```
markpublish init [ZIELORDNER] [--title "Titel"]
```

Der Stumpf ist absichtlich klein, weil er dazu da ist, gelöscht zu werden — spätestens beim ersten echten Kapitel. Die Referenz liegt deshalb nicht im Stumpf, sondern in `markpublish cheatsheet`; sowohl die YAML als auch das Kapitel verweisen darauf. Cover und Inhaltsverzeichnis sind abgeschaltet, weil ein einseitiger Entwurf beides nicht braucht.

3.3 `markpublish cheatsheet`

Rendert die mitgelieferte Kurzreferenz: Seite 1 die Schlüssel der `markpublish.yaml`, Seite 2 die Grundlagen zu Themes und Templates.

```
markpublish cheatsheet [--lang CODE] [--target pdf|html|all] [--output PFAD] [--theme NAME]
```

3.4 markpublish manual

Rendert dieses Handbuch.

```
markpublish manual [--lang CODE] [--target pdf|html|all] [--output PFAD] [--theme NAME]
```

3.4.1 Gemeinsames Verhalten der beiden Dokumentbefehle

Beide Quellen liegen im Paket und werden bei jedem Aufruf neu gerendert. Damit passt das Ergebnis immer zur installierten Version, und ein erfolgreicher Lauf ist zugleich der Nachweis, dass die Rendering-Kette funktioniert — unter Windows also, dass WeasyPrint seine GTK-Laufzeit findet. Geschrieben wird nur das fertige Dokument, und zwar ins aktuelle Verzeichnis; Quelldateien landen nie im Projekt des Nutzers.

OPTION	STANDARD	BEDEUTUNG
<code>--lang</code> , <code>-l</code>	Systemsprache	Welche Übersetzung gerendert wird. <code>markpublish manual --lang de</code>
<code>--target</code> , <code>-t</code>	<code>pdf</code>	<code>pdf</code> , <code>html</code> oder <code>all</code>
<code>--output</code> , <code>-o</code>	aktuelles Verzeichnis	Datei oder Verzeichnis
<code>--theme</code>	mitgeliefert	Rendert in einem eigenen Theme

`--lang` wählt die **Quelle**, nicht bloß die Beschriftungen — jede Übersetzung bringt ihre eigene `language:` mit und zieht damit die passenden statischen Texte von selbst nach.

Ohne Angabe entscheidet die Sprache Ihres Systems. Diese beiden Dokumente richten sich an die Person vor dem Rechner und nicht an ein Publikum; deren Sprache ist damit die beste verfügbare Vermutung. Gibt es dafür keine Übersetzung, erscheint kommentarlos die englische — verlangt wurde ja nichts. Ein ausdrückliches `--lang fr` dagegen wird gemeldet, statt still einen englischen Rahmen um eine französische Erwartung zu setzen.

Ohne `--theme` oder `--templates-dir` ignorieren beide Befehle bewusst User- und Projekt-Themes und rendern im mitgelieferten. Sonst zöge ein `templates/` im Arbeitsverzeichnis — mit `export-template` angelegt und noch mitten in der Anpassung — die Referenz mit sich: ein fehlendes Label ließe den

Build abbrechen, ausgerechnet in dem Moment, in dem jemand nachschlagen will, wie Labels funktionieren.

Fällt der PDF-Weg mangels GTK aus, liefert `--target html` dieselben Dokumente ohne WeasyPrint.

3.5 markpublish build

Kompiliert eine `markpublish.yaml`-Konfiguration.

```
markpublish build [CONFIG_FILE] [OPTIONEN]
```

3.5.1 Argumente & Optionen:

- `CONFIG_FILE` (*optional*, *Standard*: `markpublish.yaml`): Pfad zur YAML-Datei.
- `--target / -t` (*Standard*: `pdf`): Zielformat (`pdf`, `html` oder `all`).
- `--output / -o`: Benutzerdefinierter Ausgabepfad (Datei oder Verzeichnis).
- `--templates-dir`: Spezifischer Pfad zu einem gemeinsamen Template-Verzeichnis.

3.5.2 Beispiele:

```
# Standard-Build aus aktuellem Verzeichnis
markpublish build

# HTML-Version in ein bestimmtes Zielverzeichnis ausgeben
markpublish build markpublish.yaml -t html -o dist/

# Benutzerdefiniertes Template-Verzeichnis nutzen
markpublish build --templates-dir /shared/company-templates
```

Hinweis

`build` hat bewusst kein `--lang`. Die Sprache eines Dokuments gehört in seine eigene `markpublish.yaml`, neben `title` und `author`: sie beschreibt, in welcher Sprache das Dokument *geschrieben* ist. Ein Override von der Kommandozeile würde lediglich die siebzehn statischen Beschriftungen um unveränderten Fließtext herum austauschen.

3.6 markpublish templates

Zeigt eine tabellarische Übersicht aller Templates auf dem System und deren Prioritätsstatus:

```
markpublish templates
markpublish templates --target pdf
```

3.7 markpublish export-template

Kopiert das eingebaute Standard-Template in Ihr lokales Arbeitsverzeichnis:

```
markpublish export-template default templates
```

3.8 markpublish labels

Zeigt, welcher statische Text am Ende gilt und aus welcher Ebene der Label-Kaskade er stammt. Nützlich, sobald ein Theme oder ein Dokument eigene Texte mitbringt und nicht mehr offensichtlich ist, welche Ebene gewinnt.

```
markpublish labels                # alle Schlüssel, Zielformat PDF
markpublish labels --target html  # Kaskade für die HTML-Ausgabe
markpublish labels --overridden   # nur überschriebene Texte
markpublish labels pfad/zu/markpublish.yaml
```

3.8.1 Argumente & Optionen

PARAMETER	TYP	STANDARD	BESCHREIBUNG
<code>config_file</code>	Pfad	<code>markpublish.yaml</code>	Pfad zur Konfigurationsdatei
<code>--target, -t</code>	String	<code>pdf</code>	Zielformat, dessen Kaskade gezeigt wird (<code>pdf</code> oder <code>html</code>)
<code>--templates-dir</code>	Pfad	-	Alternativer Template-Ordner
<code>--overridden</code>	Flag	<code>aus</code>	Nur Texte anzeigen, die das Theme ändert

Die Spalte *Source* nennt die Ebene: `i18n.yaml (de)` für den Programmstandard oder den Pfad einer Theme- bzw. Zielformat- `i18n.yaml` . Unter der Tabelle steht, in welchen Dateien nach Overrides gesucht wurde und welche existieren.

 Tipp

`--override` beantwortet die häufigste Frage direkt: *Was weicht in diesem Projekt überhaupt vom Standard ab?*

KAPITEL 4

Konfigurations-Leitfaden

Deklarative Definition von Dokumentenmetadaten, Kapiteln und Hierarchien.

AUF EINEN BLICK

4.1 Das document-Objekt	16
4.2 Kapitel definieren	16
4.3 Hierarchische Unterkapitel	17
4.4 Übergeordnete Abschnitte (Parts / Blöcke)	17

4 Konfigurations-Leitfaden

Die Datei `markpublish.yaml` ist das zentrale Steuerungsdokument jeder Publikation. Sie gliedert sich in drei Teile: `document` für Metadaten und globale Schalter, `theme` für die Auswahl des Designs und `chapters` für den Aufbau des Dokuments.

4.1 Das `document`-Objekt

Unter `document:` werden alle globalen Metadaten und Schalter hinterlegt:

```
document:
  title: "markpublish Benutzerhandbuch"
  subtitle: "Moderne PDF- und HTML-Dokumentenerstellung"
  summary: "Kurze Zusammenfassung für Deckblatt und Metadaten."
  author: "Frank Winter"
  organisation: "WASDCAT Games"
  date: "auto"                # "auto" (Tagesdatum) oder festes Datum "2026-08-31"
  version: "1.0.0"           # optional - ohne Angabe entfällt das Feld
  language: "de"             # Beschriftungen und Datumsformat; ohne Angabe die Systemsprache

# Globale Schalter
cover: true                  # Deckblatt an/aus
document_toc: 2              # Großes Verzeichnis, zwei Ebenen tief
autonum_type: "decimal"      # "decimal", "roman", "legal", "none"
chapter_toc: 2               # Vorgabe für die Kapitel-Trennseiten
header: true                 # Laufende Kopfzeile
footer: true                 # Laufende Fußzeile
```

4.1.1 Automatische Datumsauflösung

Wird `date: "auto"` oder `date: "today"` angegeben, setzt `markpublish` das aktuelle Datum im passenden Sprachformat ein:

- `language: "de"` → 31.08.2026
- `language: "en"` → 2026-08-31

4.1.2 Zusätzliche Metadatenfelder

Beliebige zusätzliche Schlüssel (z. B. `status: "Entwurf"`, `department: "IT-Architektur"`) können frei definiert werden und stehen in allen Jinja2-Templates über `document.<feldname>` zur Verfügung.

4.2 Kapitel definieren

Ein einfaches Kapitel wird mit `file:`, optionalem `title:` und `summary:` angegeben:


```
chapters:
- file: "chapters/01_intro.md"
  title: "Einleitung"
  summary: "Zielsetzung des Leitfadens."
  break_before: "divider"      # Eigene Trennseite vor dem Kapitel
  chapter_toc: "none"          # Kein kapitelweises Mini-TOC
```

Alle Pfade sind relativ zur `markpublish.yaml`, nicht zum Arbeitsverzeichnis der Shell.

4.3 Hierarchische Unterkapitel

Um ein Unterkapitel zu definieren, rücken Sie einfach weitere Kapitel unter `chapters:` ein:

```
chapters:
- file: "chapters/02_architecture.md"
  title: "Systemarchitektur"
  break_before: "divider"
  chapter_toc: 2                # Mini-TOC bis Überschriftstiefe 2
  chapters:
    - file: "chapters/02_1_backend.md"
      title: "Backend Services"
    - file: "chapters/02_2_frontend.md"
      title: "Frontend Client"
      break_before: "none"      # Läuft ohne Umbruch weiter
```

`markpublish` nummeriert die Unterkapitel automatisch konsistent als `2.1` und `2.2`. Die Verschachtelung ist rekursiv — eine feste Obergrenze für die Tiefe gibt es nicht.

Ohne Angabe beginnt jedes Kapitel oben auf einer neuen Seite, auch ein Unterkapitel. Wie weit ein Kapitel abgesetzt wird, steuert durchgehend `break_before:` (`page`, `divider`, `none`); die Werte sind in Anhang A beschrieben.

4.4 Übergeordnete Abschnitte (Parts / Blöcke)

Für Hauptabschnitte oder Anhangsblöcke, die mehrere Kapitel umfassen, verwenden Sie das Schlüsselwort `part:`:

```
chapters:
- part: "Anhänge"
  summary: "Ergänzende Tabellen und Referenzen."
  break_before: "divider"      # Große Trennseite für den gesamten Anhang
  autonum: "none"              # Keine vorangestellte Ziffer
  document_toc: 1              # Anhänge nur mit Titel ins Inhaltsverzeichnis
  chapters:
    - file: "chapters/appendix_a.md"
      title: "Anhang A: Referenz"
    - file: "chapters/appendix_b.md"
      title: "Anhang B: Glossar"
```

Ein Part trägt selbst keinen Text: er gruppiert die Kapitel unter sich und bekommt eine eigene Trennseite. Der Part-Titel (hier *“Anhänge”*) wird automatisch in die laufende Kopfzeile der Einzelseiten übernommen.

KAPITEL 5

Das Template- und Design-System

3-stufige Auflösungshierarchie, CSS Paged Media und 2-zeilige Kopf-/Fußzeilen.

AUF EINEN BLICK

5.1 Theme-basierte Verzeichnisstruktur	20
5.2 Die 3-stufige Auflösungs-Hierarchie	20
5.3 Statische Texte und Sprache	21
5.4 Kopf- und Fußzeilen	26

5 Das Template- und Design-System

Das Template-System von `markpublish` ermöglicht vollständige visuelle Anpassbarkeit bei gleichzeitiger Wartbarkeit.

5.1 Theme-basierte Verzeichnisstruktur

Templates sind nach Theme gruppiert, darunter liegt das Ausgabeformat. So gehören alle Dateien eines Designs zusammen und ein Theme lässt sich als Ganzes kopieren, weitergeben oder versionieren:

```
templates/  
└─ default/                                # Theme-Name  
    └─ pdf/  
        └─ layout.html                    # HTML-Grundgerüst  
        └─ styles.css                    # CSS Paged Media & Kopf-/Fußzeilen  
        └─ fonts/                        # Mitgelieferte Schrift (Open Sans, variabel) + OFL.txt  
        └─ cover.html                    # Deckblatt-Template  
        └─ part_divider.html              # Trennseite für übergeordnete Parts  
        └─ chapter_divider.html          # Trennseite für Kapitel mit Mini-TOC  
        └─ toc.html                      # Globales Inhaltsverzeichnis  
    └─ html/  
        └─ layout.html                    # Standalone Web-Layout  
        └─ styles.css                    # Screen/Responsive Stylesheet  
        └─ ...
```

5.2 Die 3-stufige Auflösungs-Hierarchie

Beim Suchen nach einem Template (z. B. `default/pdf`) gilt folgende strikte Priorität:

1. User-Verzeichnis (`~/markpublish/templates/default/pdf/`)
| (höchste Priorität)
▼
2. Gemeinsamer / Projekt-Ordner (`<templates_dir>/default/pdf/`)
|
▼
3. Paket Built-in (im `markpublish` Python-Paket)

5.2.1 Konfiguration des gemeinsamen Template-Ordnern

Sie können den gemeinsamen Template-Pfad auf 4 Wegen festlegen: 1. **CLI-Parameter:** `markpublish build --templates-dir /pfad/zu/templates` 2. In `markpublish.yaml`: `templates_dir: "./custom-templates"` 3. **Umgebungsvariable:** `MARKPUBLISH_TEMPLATES_DIR=/pfad/zu/templates` 4. **Automatischer Fallback:** `./templates` im Projektordner.

5.3 Statische Texte und Sprache

Die festen Beschriftungen — Überschrift des Inhaltsverzeichnisses, Kapitel-Marken, Cover-Labels, Seitenzahl-Fußzeile, Callout-Titel — stehen nicht im Template, sondern in `i18n.yaml`-Dateien. Welche Sprache gilt, bestimmt `document.language`:

```
document:
  title: "User Guide"
  language: "en"      # steuert Beschriftungen, Callout-Titel und Datumsformat
```

Mitgeliefert sind `de` und `en`. Regionale Formen werden zugeordnet (`de-AT` → `de`), eine unbekannte Sprache fällt auf Englisch zurück. Fehlt `language`, ganz, gilt die Systemsprache — `markpublish init` schreibt sie gleich hin, damit ein Dokument auf jedem Rechner gleich gebaut wird.

Hinweis

`i18n` bezeichnet die Quellen — die Dateien und den YAML-Eintrag, jeweils über alle Sprachen. **labels** ist das daraus aufgelöste Ergebnis für *ein* Dokument in *einer* Sprache: das, was im Template unter `{{ labels.chapter }}` ankommt.

5.3.1 Die 3-stufige i18n-Kaskade

Alle drei Ebenen sind **identisch aufgebaut**: Sprachcode auf oberster Ebene, darunter die Texte. Jede tiefere Ebene überschreibt die höher liegende — und zwar **nur die Schlüssel, die sie tatsächlich setzt**. Alles andere bleibt, wie es weiter oben steht:

- | | |
|---------------|---|
| 1. Programm | <code>markpublish/i18n.yaml</code> |
| | vollständig, <code>de</code> + <code>en</code> |
| ▼ | |
| 2. Theme | <code><templates>/<theme>/i18n.yaml</code> |
| | gilt für alle Zielformate des Themes |
| ▼ | |
| 3. Zielformat | <code><templates>/<theme>/<target>/i18n.yaml</code> |
| | nur für PDF bzw. nur für HTML |

Ebene 1 ist die einzige, die vollständig sein muss. Sie legt zusätzlich Englisch unter die Dokumentsprache, damit jeder Programmtext garantiert auflöst. Die Ebenen 2 und 3 sind reine Overrides.

Statische Texte werden **ausschließlich im Theme** definiert. Eine Dokumentenebene gibt es nicht: `document.i18n` in der `markpublish.yaml` wird abgelehnt, mit Hinweis auf die Theme-Datei. Wer die Texte eines Dokuments ändern will, gibt ihm sein eigenes Theme — `markpublish export-template` legt es an.

! Wichtig

Die Ebenen 2 und 3 stammen immer aus **genau einem** Theme: welches gilt, entscheidet vorher die Template-Auflösung (User > Projekt > Paket). Ein Projekt-Theme erbt **nicht** die Texte des gleichnamigen Paket-Themes — gemischt wird nur das eine gefundene Theme mit dem Programmstandard.

! Wichtig

Die Ebenen 2 und 3 greifen **nur für die gewählte Sprache**. Ein Theme, das einen `en:`-Block definiert, verändert eine deutsche Ausgabe nicht — sonst würden englische Theme-Texte in fremdsprachige Dokumente durchschlagen.

5.3.2 Das gemeinsame Format

```
# templates/mytheme/i18n.yaml
de:
  part: "Abschnitt"
  chapter_toc_title: "Auf dieser Seite"
en:
  part: "Section"
  chapter_toc_title: "On this page"
```

Der Sonderschlüssel `"*"` gilt für **jede** Sprache und wird vor dem sprachspezifischen Block angewendet — für Begriffe, die unabhängig von der Dokumentsprache gleich heißen:

```
"*":
  version: "Rev."      # in jeder Sprache "Rev."
de:
  part: "Abschnitt"
```

Wer nur eine Sprache pflegt, darf die Sprachebene auch weglassen; die flache Form ist gleichbedeutend mit `"*"`:

```
part: "Abschnitt"      # entspricht: "*" : \n part: "Abschnitt"
```

Regionale Blöcke schlagen den Basis-Block: bei `language: "de-AT"` gewinnt `de-at:` über `de:`. Fehlt eine `i18n.yaml`, ist das kein Fehler — ein Theme ohne eigene Texte ist der Normalfall. Ist sie vorhanden, aber fehlerhaft, bricht der Build mit Angabe der Datei ab, statt still die Standardtexte zu verwenden.

5.3.3 Beispiel: PDF und HTML unterschiedlich beschriften

```
templates/mytheme/  
├─ i18n.yaml           de: chapter: "Kapitel"  
├─ pdf/  
│   └─ i18n.yaml       de: chapter: "Kap."      ← nur im PDF  
└─ html/  
    └─ i18n.yaml        (leer → erbt "Kapitel")
```

Das mitgelieferte Theme `default` bringt alle drei Dateien als **auskommentiertes Muster** mit: `templates/default/i18n.yaml`, `default/pdf/i18n.yaml` und `default/html/i18n.yaml`. Sie sind bewusst wirkungslos — das Standard-Theme soll exakt wie der Programmstandard aussprechen. Kommentieren Sie aus, was Sie ändern möchten. `markpublish export-template` kopiert diese Dateien mit — auch die `i18n.yaml` der Theme-Ebene, die neben den Zielformat-Ordern liegt.

5.3.4 Freie Labels: eigene Texte des Templates

Ein Theme darf **eigene Schlüssel** definieren, die das Programm nicht kennt — für die statischen Texte des Templates selbst. Der Aufbau ist derselbe, der Zugriff ebenso:

```
# templates/mytheme/i18n.yaml  
de:  
  imprint_title: "Impressum"  
  disclaimer: "Alle Angaben ohne Gewähr."  
en:  
  imprint_title: "Imprint"  
  disclaimer: "All information without guarantee."
```

```
<!-- templates/mytheme/html/Layout.html -->  
<footer>{{ labels.imprint_title }}</footer>
```

⚠️ Warnung

Für freie Labels gibt es **keinen Programmstandard**, der einspringen könnte. Deshalb gilt hier eine strikte Regel: Ein Label, das ein Template notiert, **muss** in der Kaskade auflösen. Tut es das nicht, bricht der Build ab und nennt Schlüssel, Fundstelle mit Zeilennummer, Dokumentsprache und die durchsuchten Dateien:

```
Label 'imprint_title' ist im Template notiert, aber in keiner i18n-Ebene definiert.  
Dokumentsprache: de  
Fundstelle:  
  templates/mytheme/html/layout.html:108  
Gesucht in:  
  templates/mytheme/html/i18n.yaml (vorhanden)  
  templates/mytheme/i18n.yaml      (vorhanden)  
  markpublish/i18n.yaml            (vorhanden)
```

Pflegen Sie also jede Sprache, die Sie ausliefern, oder legen Sie den Schlüssel unter `"*"` ab. Ein leerer Text im fertigen PDF fällt niemandem auf — ein Abbruch schon.

Geprüft werden die Template-Quellen, nicht der Renderlauf: auch ein Label in einem Zweig, den genau dieses Dokument nicht durchläuft, wird gemeldet. `styles.css` zählt mit, da es durch dieselbe Jinja-Umgebung läuft.

5.3.5 Die aufgelöste Tabelle ansehen

Bei drei Ebenen ist nicht immer offensichtlich, woher ein Text kommt. `markpublish labels` zeigt das Ergebnis samt Herkunft:

```
markpublish labels          # alle Schlüssel, Ziel PDF  
markpublish labels --target html # Kaskade für die HTML-Ausgabe  
markpublish labels --overrides # nur das, was vom Theme kommt
```


5.3.6 Verfügbare Schlüssel

SCHLÜSSEL	DE	EN
<code>toc_title</code>	Inhaltsverzeichnis	Table of Contents
<code>toc_sidebar</code>	Inhalt	Contents
<code>chapter_toc_title</code>	Inhalt dieses Kapitels	In this chapter
<code>chapter</code>	Kapitel	Chapter
<code>part</code>	Teil	Part
<code>author</code>	Autor	Author
<code>status</code>	Status	Status
<code>version</code>	Version	Version
<code>date</code>	Datum	Date
<code>copyright</code>	Copyright	Copyright
<code>page</code>	Seite	Page
<code>page_of</code>	von	of
<code>alert_note</code>	Hinweis	Note
<code>alert_tip</code>	Tipp	Tip
<code>alert_important</code>	Wichtig	Important
<code>alert_warning</code>	Warnung	Warning
<code>alert_caution</code>	Achtung	Caution

Die `alert_*`-Titel entstehen bereits beim Markdown-Parsen, nicht erst im Template — die Kaskade wird dorthin durchgereicht, ein `alert_note` im Theme wirkt also auch im Callout.

Eine neue Sprache legen Sie an, indem Sie in `markpublish/i18n.yaml` einen vollständigen Block ergänzen — oder, ohne das Paket anzufassen, auf Ebene 2 oder 3 alle Schlüssel unter dem gewünschten Sprachcode setzen.

💡 Tipp

In eigenen Templates greifen Sie mit `{{ labels.chapter }}` auf das Ergebnis zu — auch in `styles.css`, das durch dieselbe Jinja-Umgebung läuft. So ist die Fußzeile `"{{ labels.page }}" counter(page) "` `{{ labels.page_of }}" counter(pages)` gebaut.

5.4 Kopf- und Fußzeilen

Kopf- und Fußzeile sind **Running Elements**: ein Block im `layout.html` bekommt

`position: running(name)` und wird damit aus dem Textfluss genommen; die `@page`-Regel setzt ihn über `content: element(name)` in die Margin-Box ein.

Der Unterschied zu einer `content:`-Zeichenkette ist wesentlich: In der Margin-Box steht dadurch **echtes Markup**. Damit sind beliebig viele Zeilen möglich, jede mit eigener Auszeichnung — eine Zeichenkette kennt nur eine Formatierung für alles.

```
<!-- layout.html -->
<div class="page-header-runner">
  <div class="hf-left">
    <div class="hf-doc-title">{{ document.title }}</div>
    <div class="hf-doc-subtitle">{{ document.subtitle }}</div>
  </div>
  <div class="hf-right"><span class="hf-section"></span></div>
</div>
```

```

/* styles.css */
.page-header-runner {
  position: running(pageheader);
  display: flex;
  justify-content: space-between;
  align-items: flex-start; /* rechte Spalte bleibt oben */
}
.hf-doc-title { font-weight: 700; }
.hf-section::before { content: string(current-section); }

@page {
  @top-left {
    content: element(pageheader);
    width: 100%;
    vertical-align: top;
    margin-top: 12mm; /* Abstand zur Papierkante */
    padding-bottom: 0;
    border-bottom: 0.5pt solid #cbd5e1;
    margin-bottom: 12mm; /* Abstand zum Inhalt */
  }
}

```

5.4.1 Geometrie

Von der Papierkante nach innen:

Kante — margin — Zeilen (top-aligned) — Linie — margin — Inhalt
 12 mm 12 mm

Beide Abstände sind bewusst gleich groß: eine Kopfzeile, die dicht über dem Text sitzt, liest sich als Teil des Satzspiegels statt als Seitenfurnitur.

Beide Spalten sitzen in einem Flex-Container mit `align-items: flex-start`. Die rechte Spalte beginnt deshalb auf der Höhe der **ersten** linken Zeile, auch wenn links drei Zeilen stehen und rechts nur eine.

Wichtig

Der Abstand zum Inhalt kommt aus `margin-bottom`, nicht aus `padding-bottom`. In CSS liegt der Rahmen **außerhalb** des Paddings — ein `padding-bottom` würde die Trennlinie vom Text wegschieben und an den Inhalt drücken. Gewollt ist das Gegenteil: Linie direkt am Text, Abstand danach.

⚠️ Warnung

Eine Margin-Box **schiebt den Inhalt nicht**. Ihre Höhe ist durch den Seitenrand gedeckelt; zusätzliche Zeilen laufen aus der Seite heraus, statt den Satzspiegel zu verkleinern. Der Seitenrand wird deshalb in `styles.css` aus der Zeilenzahl gerechnet:

```
margin-top = Rand zur Kante + Zeilen × Zeilenhöhe + Linienstärke + Abstand zum Inhalt
```

Wer in `layout.html` eine Zeile ergänzt, zieht `hf_header_lines` bzw. `hf_footer_lines` in `styles.css` mit.

5.4.2 Standardbelegung des Themes

Kopfzeile	Dokumenttitel (fett) Untertitel	Kapiteltitel
Fußzeile	Copyright	Version 1.0.0 01.09.2026 Seite X von Y

Untertitel und Version sind optional. Fehlt der Untertitel, hat die Kopfzeile nur eine Zeile und der Satzspiegel rückt entsprechend nach oben. Fehlt die Version, entfällt sie samt Trenner — in der Fußzeile steht dann nur das Datum, und auf dem Deckblatt fehlt das Feld ganz.

Der Kapiteltitel kommt aus `string(current-section)`, das die Kapitel- `<article>` per `string-set` setzen; `counter(page)` funktioniert innerhalb des Running Elements ebenso wie in einer Margin-Box. Die Schalter `document.header` und `document.footer` blenden den jeweiligen Block ab; auf Deck- und Trennseiten ist er ohnehin abgeschaltet.

KAPITEL 6

Markdown-Features & Syntax

Tabellen, Admonitions/Callouts, Pygments Syntax-Highlighting und Tasklisten.

6 Markdown-Features & Syntax

markpublish beinhaltet eine leistungsfähige Markdown-Engine mit voller Unterstützung moderner Dokumentations-Elemente.

6.1 Code-Blöcke & Syntax Highlighting

Quellcode wird durch Pygments hervorgehoben:

```
from markpublish.config.loader import load_config
from markpublish.markdown.engine import MarkdownPipeline

# Manifest Laden
config = load_config("markpublish.yaml")
print(f"Building: {config.document.title}")
```

6.2 Tabellen

Tabellen werden mit sauberem CSS-Zebrawuster und Seitenumbruchschutz gerendert:

PARAMETER	TYP	STANDARD	BESCHREIBUNG
title	String	<i>Pflicht</i>	Haupttitel des Dokuments
author	String	None	Name des Autors
status	String	None	Dokumentstatus (z.B. "Freigegeben", "Draft")
copyright	String	None	Copyright-Vermerk (z.B. "© 2026 Frank Winter")
version	String	null	Versionskennung. Ohne Angabe entfällt das Feld auf dem Deckblatt; ist sie gesetzt, steht sie in der Fußzeile links neben dem Datum

6.3 GitHub-Style Callout-Boxen (Alerts)

markpublish unterstützt die gängige **GitHub-Alert-Syntax** mit fünf semantischen Typen. Die Icons stammen dabei direkt aus dem Template:

Hinweis

Dies ist eine informative Notiz mit blauem Akzent und passendem Info-Icon.

Tipp

Nutzen Sie `markpublish build --target all`, um PDF und HTML parallel in einem Durchgang zu erstellen.

Wichtig

GitHub-Callouts werden automatisch anhand der Dokumentensprache lokalisiert (z. B. *Hinweis*, *Tipp*, *Wichtig*, *Warnung*, *Achtung*).

Warnung

Achten Sie bei relativen Bildpfaden darauf, dass diese relativ zur jeweiligen Markdown-Datei liegen.

Achtung

Fehlerhafte YAML-Einrückungen führen zu Abbruchfehlern beim Parsen des Manifests.

Eigener Titel

Sie können nach dem Alert-Typ auch einen individuellen Titel angeben, der die Standardbeschriftung überschreibt.

Alternativ wird auch weiterhin die klassische `!!! note`-Syntax unterstützt.

6.4 Tasklisten

- ☒ Projekt initialisieren
- ☒ Kapitel schreiben
- ☐ Dokument veröffentlichen

Anhang A: YAML-Schema-Referenz

Vollständige Übersicht aller Konfigurationsoptionen in `markpublish.yaml`.

Dokumenten-Eigenschaften (document)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
title	String	<i>Pflicht</i>	Titel der Publikation
subtitle	String	null	Untertitel
summary	String	null	Zusammenfassung für Deckblatt
author	String	null	Name des Autors
status	String	null	Dokumentstatus (z. B. "Entwurf", "Freigegeben")
copyright	String	null	Copyright-Angabe (z. B. "© 2026 Frank Winter")
date	String	"auto"	Datum oder "auto" für Tagesdatum
version	String	null	Versionskennung. Ohne Angabe entfällt das Feld auf dem Deckblatt; ist sie gesetzt, steht sie in der Fußzeile links neben dem Datum
language	String	Systemsprache	ISO-Sprachcode (de , en , ...). Steuert Template-Beschriftungen, Callout-Titel und Datumsformat
cover	Bool	true	Deckblatt aktivieren/deaktivieren
document_toc	String/ Int	full	Das große Verzeichnis: none , full oder eine Tiefe. Vorgabe für die Kapitel
autonum_type	String	"decimal"	"decimal" , "roman" , "legal" , "none" . Wurzel für chapters.autonum
chapter_toc	String/ Int	none	Die kleinen Verzeichnisse: none , full oder eine Tiefe. Vorgabe für die Kapitel
header	Bool	true	Laufende Kopfzeile aktivieren (Aufbau im Theme)
footer	Bool	true	Laufende Fußzeile aktivieren (Aufbau im Theme)

Kapitel- und Part-Eigenschaften (`chapters`)

SCHLÜSSEL	TYP	STANDARD	BESCHREIBUNG
<code>file</code>	String	<code>null</code>	Pfad zur Markdown-Datei
<code>title</code>	String	<code>null</code>	Überschreibt den Titel der Datei
<code>summary</code>	String	<code>null</code>	Zusammenfassung für Trennseite
<code>part</code>	String	<code>null</code>	Deklariert einen übergeordneten Part
<code>break_before</code>	String	<code>page</code>	Wie das Kapitel abgesetzt wird: <code>page</code> , <code>divider</code> oder <code>none</code> . Nur PDF — das mitgelieferte HTML-Theme kennt weder Seiten noch Trennseiten
<code>chapter_toc</code>	String/ Int	<code>none</code>	Das kleine Verzeichnis auf der Trennseite des Kapitels. Also ebenfalls nur PDF
<code>document_toc</code>	String/ Int	<code>full</code>	Der Beitrag zum großen Verzeichnis vorn im Dokument. Wird nach unten vererbt
<code>autonom</code>	String	<code>null</code>	Nummerierungsstil für diesen Zweig. Wird nach unten vererbt
<code>chapters</code>	Liste	<code>[]</code>	Verschachtelte Unterkapitel

`break_before` — wie ein Kapitel abgesetzt wird

Ein Schlüssel mit drei Werten, keine zwei unabhängigen Schalter:

WERT	WIRKUNG
<code>page</code>	Standard. Das Kapitel beginnt oben auf einer neuen Seite
<code>divider</code>	Dem Kapitel geht eine eigene Trennseite voraus
<code>none</code>	Das Kapitel läuft im Fließtext weiter

Dass es *ein* Schlüssel ist, hat einen Grund: die drei Werte sind eine Achse, nicht drei Fragen. Als zwei Booleans ließe sich „Trennseite, aber kein Seitenumbruch“ hinschreiben — ein Zustand, den es nicht gibt, weil eine Trennseite immer umbricht. Eine Angabe, die man notieren kann und die nichts bewirkt, ist eine Fehlerquelle ohne Gegenwert.

`page` ist die Erwartung an ein gesetztes Dokument. Wer kurze Abschnitte durchlaufen lassen will — Merkbblätter, Referenzkarten, eng gesetzte Anhänge — setzt am einzelnen Kapitel `break_before: "none"`; die Vorgabe lässt sich auch im Frontmatter der Markdown-Datei überschreiben.

Bei `divider` fallen der Umbruch des Kapitels und der der Trennseite auf dieselbe Stelle und verschmelzen zu einem — es entsteht kein Leerblatt. Vor dem ersten Kapitel greift die Regel nicht.

Ein unbekannter Wert bricht den Build ab, statt still auf `page` zurückzufallen: aus einem vertippten `divder` würde sonst klaglos ein normaler Seitenumbruch, und die fehlende Trennseite fände man erst beim Durchblättern des fertigen PDFs.

`chapter_toc` und `document_toc` — zwei Verzeichnisse, ein Vokabular

Ein Dokument hat zwei Inhaltsverzeichnisse, und für jedes gibt es ein Paar aus Vorgabe und Einzelfall:

VERZEICHNIS	VORGABE UNTER <code>DOCUMENT:</code>	AM KAPITEL
Das große vorn im Dokument	<code>document_toc</code>	<code>document_toc</code>
Das kleine auf der Kapitel-Trennseite	<code>chapter_toc</code>	<code>chapter_toc</code>

Alle vier Schlüssel nehmen dieselben drei Schreibweisen:

WERT	BEDEUTUNG
<code>none</code>	Kommt in diesem Verzeichnis gar nicht vor
<code>full</code>	Jede Ebene
<i>Zahl</i>	Bis zu dieser Tiefe, gezählt ab der Kapitelüberschrift

Die Tiefe zählt **innerhalb des Kapitels**: 1 ist die Kapitelüberschrift selbst, 2 die Ebene darunter.

`document_toc: 1` nimmt das Kapitel also ins große Verzeichnis auf, seine Zwischenüberschriften aber nicht. `chapter_toc: 2` listet auf der Trennseite genau die Ebene unterhalb der Kapitelüberschrift.

Für den häufigsten Fall genügen damit zwei Zeilen im `document`-Block:

```
document:
  document_toc: 2      # großes Verzeichnis, zwei Ebenen tief
  chapter_toc: 2       # kleine Verzeichnisse ebenso
```

`document_toc: "none"` lässt das große Verzeichnis ganz weg; die Angaben an den Kapiteln sind dann gegenstandslos.

Vererbung

`document_toc` wird nach unten vererbt: an einem Part gesetzt, gilt er für alle Kapitel darunter, und ein Kapitel gibt ihn an seine Unterkapitel weiter. Für den häufigsten Fall — ein Anhang, dessen innere Gliederung das Verzeichnis nur aufbläht — genügt damit eine Zeile:

```
- part: "Anhänge"
  document_toc: 1
  chapters:
    - file: "chapters/07_appendix_yaml_spec.md"
      title: "Anhang A: YAML-Schema-Referenz"
    - file: "chapters/08_appendix_troubleshooting.md"
      title: "Anhang B: Troubleshooting"
```

Ein einzelnes Kapitel schlägt das Geerbte — auch zurück auf `fu11`. Genau dafür gibt es das Schlüsselwort: ohne es müsste man eine willkürlich große Zahl hinschreiben, um „doch wieder alles“ zu sagen.

`chapter_toc` wird **nicht** von Kapitel zu Unterkapitel gereicht. Es beschreibt die Trennseite genau dieses Kapitels, und die hat jedes Kapitel für sich; ohne eigene Angabe gilt schlicht `document.chapter_toc`.

Was gekürzt wird — und was nicht

Der Fließtext bleibt unberührt: die Zwischenüberschriften stehen weiterhin im Kapitel, samt Nummerierung und Sprungzielen. Gekürzt wird ausschließlich das Verzeichnis. Ein Unterkapitel behält dabei seinen eigenen Eintrag — vererbt wird die Tiefe relativ zu jedem Kapitel, nicht über die zusammengelegte Liste.

Ein unbekannter Wert bricht den Build ab. Auch Wahrheitswerte werden abgewiesen: einem `true` sähe man die Tiefe nicht an — genau dafür gibt es `fu11`.

Nummerierung

`document.autonum_type` legt den Stil für das ganze Dokument fest; `autonum` an einem Kapitel oder Part weicht davon ab und **gibt den Wert nach unten weiter**. Ohne diese Vererbung bliebe die Angabe am Part wirkungslos: die Überschriften stehen in den Kapiteldateien, nicht im Part.

`autonum: "none"` heißt: in diesem Zweig trägt **nichts** eine Nummer — weder die Kapitelüberschrift noch die Ebenen darunter. Es gibt also auch keinen Neustart bei 1, denn innerhalb des Zweigs läuft keine Zählung, die neu beginnen könnte.

Der Dokumentzähler bleibt dabei unangetastet. Eine unnummerierte Strecke verbraucht keine Nummer:

```
chapters:
- file: "chapters/01.md"           # 1
- part: "Anhänge"
  autonum: "none"                  # Anhänge: keine Nummern
  chapters:
    - file: "chapters/a.md"
    - file: "chapters/b.md"
- file: "chapters/02.md"           # 2, nicht 4
```

Anhang B: Fehlerbehebung & FAQ

Häufige Fragen und Lösungen bei der Dokumentenerstellung.

WeasyPrint & GTK-Laufzeitumgebung auf Windows

Problem: `cannot load library 'libgobject-2.0-0'`

WeasyPrint benötigt auf Windows-Systemen die nativen C-Bibliotheken von Pango und GTK.

Lösung: `markpublish` sucht automatisch nach installierten GTK-Umgebungen (wie MSYS2, GTK3-Runtime, darktable oder Inkscape). Sollten diese fehlen, installieren Sie das offizielle GTK3-Runtime-Paket:

- Download: [GTK-for-Windows-Runtime-Environment-Installer](#)
- Alternativ via MSYS2: `pacman -S mingw-w64-x86_64-pango`

Wenn Sie nur schnell etwas nachlesen wollen: `--target html` rendert ganz ohne WeasyPrint — auch `markpublish cheatsheet --target html` und `markpublish manual --target html`.

Bilder werden im PDF nicht angezeigt

Problem: Leere Bildrahmen oder fehlende Grafiken

WeasyPrint benötigt gültige relative Pfade oder absolute File-URIs.

Lösung: Geben Sie relative Bildpfade immer relativ zu der Markdown-Datei an, die sie einbindet:

```
![Architekturdiagramm](images/architecture.png)
```

`markpublish` bettet relative Grafiken automatisch als portable Data-URIs ein (und nutzt für Dateien über 12 MB absolute Datei-URIs).

Inhaltsverzeichnis zeigt falsche Seitenzahlen

Das PDF-Rendering erfolgt in mehreren Layout-Durchläufen (Zweipass-Rendering von WeasyPrint). Stellen Sie sicher, dass alle internen Überschriften-IDs eindeutig sind — `markpublish` übernimmt diese Eindeutigkeit automatisch über seinen internen Slug-Generator.

Ein Build bricht mit einem Label-Namen ab

```
Label 'imprint_title' is used in the template but defined in no i18n level.
```

Ein Theme verwendet einen statischen Text, der in keiner Ebene der i18n-Kaskade auflöst. Der Abbruch ist Absicht und die bessere Alternative zum stillen Leerstring: ein leerer Text im fertigen PDF fällt niemandem auf, ein Abbruch schon.

Lösung: Definieren Sie den Schlüssel in der `i18n.yaml` des Themes — unter jeder Sprache, die Sie ausliefern, oder unter `"**"`, wenn er überall gleich lauten soll. Was aktuell auflöst, zeigt `markpublish labels`.

`markpublish cheatsheet` oder `manual` schlägt fehl, nachdem ich ein Theme bearbeitet habe

Beide Befehle rendern bewusst im mitgelieferten Theme und ignorieren ein `templates/`-Verzeichnis im Arbeitsordner — genau damit ein halbfertiges Theme die Referenz nicht mit sich reißen kann. Wer sie *doch* im eigenen Theme sehen will, verlangt das ausdrücklich mit `--theme NAME`; dann schlägt ein defektes Theme durch, was der Sinn des Nachfragens ist.